

Hackathon Problem Suggestion

RESUME_DRAFT_11.pdf

PDF

ET_AI_Hackathon_2026_Problem_Statements.pdf

PDF

This is my tech stack and ML part will be handled by my friend. Now suggest me which problem statement to pick which will be impactful, innovative and will be able to win us the Hackathon.

I looked at both:

- Your profile/stack: strong in **Next.js, React, TypeScript, FastAPI, Node.js, MongoDB, PostgreSQL, AI workflows, RAG, dashboards, geospatial visualizations, agentic systems.**
- The ET AI Hackathon problem statements.

RESUME_DRAFT_11

ET_AI_Hackathon_2026_Problem_St...

Given your team composition (**you handle full-stack/product/architecture and your friend handles ML**), I would rank the problem statements like this:

1. AI-Powered Urban Air Quality Intelligence (PS-5) — Best Chance to Win

This is the one I'd choose.

Why?

Most teams will build:

- AQI dashboard
- AQI prediction model
- Some maps

Judges have seen these before.

Instead, build:

"CityBrain AI"

A multi-agent platform that:

1. Predicts AQI for next 24-72 hours
2. Identifies WHY pollution will increase
3. Suggests interventions
4. Estimates impact of intervention
5. Generates reports automatically for city authorities

The statement explicitly wants source attribution, forecasting, enforcement intelligence, and citizen advisories.

ET_AI_Hackathon_2026_Problem_St... ET_AI_Hackathon_2026_Problem_St...

Why this fits your stack

You can build:

- Next.js dashboard
- Geospatial heatmaps
- Agent workflows
- RAG
- Interactive city simulation

Very quickly.

Your friend can focus on:

- AQI forecasting
 - Source attribution model
-

Winning Feature

Add:

"What-if Simulator"

Example:

"If construction activity is reduced by 30% in Ward 5, AQI improves by 18 points."

Judges love decision-support systems.

Most teams stop at prediction.

Winners provide recommendations.

Impact Score

10/10

Innovation Score

9.5/10

Demo Score

10/10

2. AI for Digital Public Safety (PS-6)

Extremely impactful.

ET_AI_Hackathon_2026_Problem_St...

Why it is strong

Digital arrest scams are exploding.

Every judge instantly understands the problem.

You can create:

- Scam detection AI
- Fraud graph intelligence
- WhatsApp assistant
- Citizen safety copilot

Winning Idea

Build:

"ScamShield AI"

User uploads:

- screenshot
- call recording
- chat message

System:

- detects scam probability
- explains why
- maps scam network
- generates complaint draft

Problem

Needs good fraud datasets.

Data may be harder than AQI.

Impact

10/10

Innovation

9/10

Feasibility

7.5/10

3. Industrial Knowledge Intelligence (PS-8)

This is secretly one of the strongest AI problems.

ET_AI_Hackathon_2026_Problem_St...

Why?

It is basically:

- RAG
- Knowledge Graph
- Agentic AI
- OCR
- Document Intelligence

which is exactly where modern AI is heading.

Why it fits your stack

You already built:

- AI Representation Optimizer
- Buildora
- RAG-like systems
- FastAPI services

So you can move fast.

Winning Version

Not just document search.

Create:

Industrial Brain

Upload:

- manuals
- inspection reports
- SOPs
- incident reports

Then ask:

Why did Pump P-101 fail three times last year?

AI reasons across all documents.

That is far more impressive than a chatbot.

Impact

8.5/10

Technical Depth

10/10

Demo

8/10

Problems I Would Avoid

PS-2 Energy Supply Chain

Interesting but:

- huge geopolitical complexity
- difficult validation
- hard to demonstrate in 24-48 hours

ET_AI_Hackathon_2026_Problem_St...

PS-3 EV Supply Chain

Very broad.

Needs:

- battery analytics
- fleet data
- supply chain intelligence

Too much scope.

ET_AI_Hackathon_2026_Problem_St...

PS-4 Data Centre EPC

Very domain-heavy.

Judges may appreciate it, but it's harder to create a wow factor quickly.

ET_AI_Hackathon_2026_Problem_St...

PS-7 Cyber Resilience

Strong technically.

ET_AI_Hackathon_2026_Problem_St...

But every cybersecurity team will build:

- anomaly detection
- threat intelligence
- SOC dashboard

Competition will be intense.

My Final Ranking

Rank	Problem	Win Probability
	Urban Air Quality Intelligence	9.5/10
	Digital Public Safety	9.2/10
	Industrial Knowledge Intelligence	8.8/10
4	Industrial Safety Intelligence	8.5/10
5	Cyber Resilience	8.3/10
6	Data Centre EPC	7.8/10
7	Energy Supply Chain	7.5/10
8	EV Supply Chain	7.2/10

If your goal is specifically "**maximize chances of winning with your current full-stack + AI team**", I would pick **PS-5 Urban Air Quality Intelligence** and build a **multi-agent city intervention platform with AQI forecasting, source attribution, intervention recommendations, and a what-if simulator**. That combination hits all five judging categories: innovation, impact, technical excellence, scalability, and UX.

ET_AI_Hackathon_2026_Problem_St...

I would like to go with this AI-Powered Urban Air Quality Intelligence for Smart City Intervention Theme: Smart Cities / Environmental Intelligence / Geospatial Analytics / Public Health
PROBLEM CONTEXT

India's air quality crisis is not a Delhi problem — it is a national urban crisis. In 2024-25, Delhi averaged an AQI of 218 (classified 'Poor' or worse for over 200 days), but the situation across other metros is nearly as severe: Mumbai recorded dangerous AQI levels on over 60 days in 2024, Kolkata averaged AQI above 150 for large parts of the winter season, and Bengaluru and Chennai — long considered relatively clean cities — have seen measurable deterioration as vehicle density

and construction activity surge. CPCB's National Air Quality data for 2024 shows that 24 of India's 50 most polluted cities are Tier 1 or Tier 2 urban centres. The Lancet Planetary Health journal estimated 1.67 million premature deaths annually from air pollution in India — a public health burden that falls disproportionately on urban populations. Despite India deploying over 900 Continuous Ambient Air Quality Monitoring Stations (CAAQMS) under the National Clean Air Programme, a 2024 CAG audit found that only 31% of cities with monitoring data had any actionable multi-agency response protocols linked to those readings. The data exists. The intelligence layer to act on it does not. City administrations need more than dashboards. They need geospatial attribution (which sources are responsible at this location, right now), predictive forecasting (what will AQI be in 24 hours at ward level), and enforcement intelligence (where to deploy inspectors for maximum impact). That combination does not exist today.

CHALLENGE STATEMENT

Build an AI-powered Urban Air Quality Intelligence platform that fuses monitoring station data, satellite imagery, mobility feeds, meteorological forecasts, and geospatial land use layers to move from reactive monitoring to proactive, evidence-based intervention — giving city administrators the tools to reduce pollution at source rather than just measure it.

WHAT YOU MAY BUILD

Participants may explore areas such as:

- Geospatial Pollution Source Attribution Engine — Multi-modal AI agent that analyses spatial-temporal AQI patterns against land use maps, traffic density, construction permits, industrial stacks, and satellite-detected thermal anomalies — attributing pollution by source category at ward or zone level with statistical confidence scores.
- Hyperlocal Predictive AQI Forecasting Agent — AI model integrating meteorological forecasts, traffic prediction, seasonal emission calendars, and atmospheric dispersion modelling to provide 24-72 hour AQI forecasts at 1km grid resolution across city boundaries — enabling intervention scheduling rather than reactive advisories.
- Enforcement Intelligence & Prioritisation Agent — Agent that correlates pollution hotspot data with registered emission sources — industries, construction sites, waste burning locations, diesel fleet movement — and generates prioritised, evidence-backed enforcement action recommendations for municipal and pollution control authorities with supporting geospatial documentation.
- Multi-City Comparative Intelligence Dashboard — Geospatial analytics layer that tracks and compares air quality trends, intervention effectiveness, and compliance metrics across multiple urban centres — enabling city administrators to learn from interventions that worked in comparable cities.
- Citizen Health Risk Advisory System — AI platform that generates ward-level health risk alerts, maps population vulnerability (hospitals, schools, outdoor workers, elderly populations) against forecast AQI, and pushes personalised advisories through mobile apps, public displays, and IVR in regional languages — Bengaluru in Kannada, Chennai in Tamil, and so on.

These examples are illustrative only.

SUGGESTED TECHNOLOGIES

- Geospatial Intelligence & Remote Sensing (Sentinel satellite, MODIS)
- Multi-Agent AI Systems
-

Real-Time IoT Sensor Data Integration (CAAQMS)

- Atmospheric Dispersion Modelling

- Predictive Analytics

- LLMs for multi-language citizen communication

EXPECTED DELIVERABLES

- Working Prototype

- Architecture Diagram

- Presentation Deck

- Demo Video

Evaluation Focus Source attribution accuracy versus ground-truth emission inventories, AQI forecast accuracy at hyperlocal resolution (RMSE versus persistence baseline), enforcement recommendation quality rated by domain experts, citizen advisory relevance and language coverage, and demonstrated reduction in response time from signal to intervention. Suggest me the idea and how to approach it

Show moreShow less

This is a very strong choice.

Most teams will build **AQI Prediction + Dashboard**.

That won't win.

To win, your solution should answer:

"AQI is going to worsen tomorrow. What exactly should the city do right now, where should they do it, how much improvement will it create, and which citizens are at risk?"

That is the intelligence layer the problem statement is asking for.

ET_AI_Hackathon_2026_Problem_St...

Winning Product Vision

Name

AERIS AI

(Air Environmental Response & Intelligence System)

Tagline:

Predict. Explain. Intervene.

Core Innovation

Don't build an AQI monitoring system.

Build an **Urban Air Quality Operating System**.

Current solutions tell cities:

AQI = 260

Your system tells cities:

AQI will reach 320 in Ward 18 within 18 hours.

47% contribution from construction activity.

31% contribution from heavy traffic.

Shut down 6 construction zones and reroute trucks.

Expected AQI improvement: 42 points.

That is a winning story.

Architecture

Agent 1: AQI Forecasting Agent

Input:

- Historical AQI
- Weather
- Temperature
- Wind speed
- Humidity
- Rainfall

Output:

```
JSON
{
  "ward": "18",
  "forecast_aqi": 312,
  "confidence": 0.91
}
```

Friend handles ML.

You handle visualization.

Agent 2: Pollution Attribution Agent

This is where innovation begins.

Inputs:

- AQI stations
- Road traffic density
- Construction permits
- Industrial locations
- Satellite thermal anomalies

Outputs:

JSON

```
{
  "traffic": 31%,
  "construction": 47%,
  "industry": 12%,
  "other": 10%
}
```

Displayed as map overlays.

This directly satisfies:

Geospatial Pollution Source Attribution Engine

from the problem statement.

ET_AI_Hackathon_2026_Problem_St...

Agent 3: Intervention Agent

This is your killer feature.

Input:

JSON

```
AQI forecast = 312
construction = 47%
traffic = 31%
```

Output:

Action Priority #1
Suspend construction permits in Zone A

Expected AQI reduction:
-22 points

Action Priority #2
Restrict diesel truck entry

Expected AQI reduction:
-11 points

No team will likely build this well.

Judges will love it.

Agent 4: What-if Simulator

This can win the hackathon.

City official asks:

What if we reduce truck movement by 30%?

AI answers:

AQI falls from 312 → 281

Health risk decreases 14%

Or

What if construction activity stops for 48 hours?

AQI falls 312 → 250

Now you're helping decisions.

Not reporting data.

Agent 5: Citizen Health Advisor

Problem statement explicitly asks for this.

ET_AI_Hackathon_2026_Problem_St...

Example:

Ward AQI = 340

AI generates:

Asthma Patients

Avoid outdoor activity.

Schools

Cancel outdoor sports.

Senior Citizens

Wear N95 masks.

Construction Workers

Limit exposure to 2 hours.

Can support:

- English
- Hindi
- Bengali
- Tamil

using LLM translation.

Dashboard Screens

Screen 1

City Overview

Map of city

Color coded AQI

Green → Red

Screen 2

Pollution Attribution

Click Ward 18

Shows:

Traffic: 31%
Construction: 47%
Industry: 12%
Other: 10%

Screen 3

Forecast View

Next:

- 24h
- 48h
- 72h

Heatmap animation.

Screen 4

Intervention Center

AI recommendations.

Priority ranking.

Expected impact.

Cost estimate.

Screen 5

What-if Simulator

Slider based.

Traffic ↓ 20%
Construction ↓ 30%

Instant prediction.

Screen 6

Citizen Risk Dashboard

Schools

Hospitals

Elderly Zones

Mapped on GIS.

Tech Stack

Perfectly aligned with your skills.

Frontend

- Next.js
 - TypeScript
 - Tailwind
 - Mapbox GL
-

Backend

- FastAPI

or

- Node + Express
-

Database

- PostgreSQL + PostGIS

or

- MongoDB

I recommend PostgreSQL + PostGIS.

AI Layer

Friend:

- XGBoost
- LightGBM
- LSTM

for AQI forecasting

Geospatial

- GeoJSON
 - OpenStreetMap
 - Mapbox
-

LLM Layer

- Gemini
- OpenAI

For:

- explanations
 - interventions
 - advisories
-

Datasets You Can Actually Use

AQI

CPCB Open Data

Weather

OpenWeather

Satellite

Sentinel-5P

Traffic

TomTom API
or mock dataset

Land Use

OpenStreetMap

Population

Indian Census

Hackathon Demo Flow

This is the pitch I'd give:

1. Open Bengaluru map
2. Show current AQI
3. Show tomorrow AQI forecast
4. Show source attribution
5. Ask AI:

Why is AQI increasing?

6. AI explains
7. Open What-if Simulator

Reduce construction by 40%

8. AQI drops
9. AI generates intervention plan
10. Citizen advisories auto-generated

That demo is memorable.

MVP vs Winner Version

Basic MVP (6/10)

- AQI Forecast
- Dashboard
- Heatmap

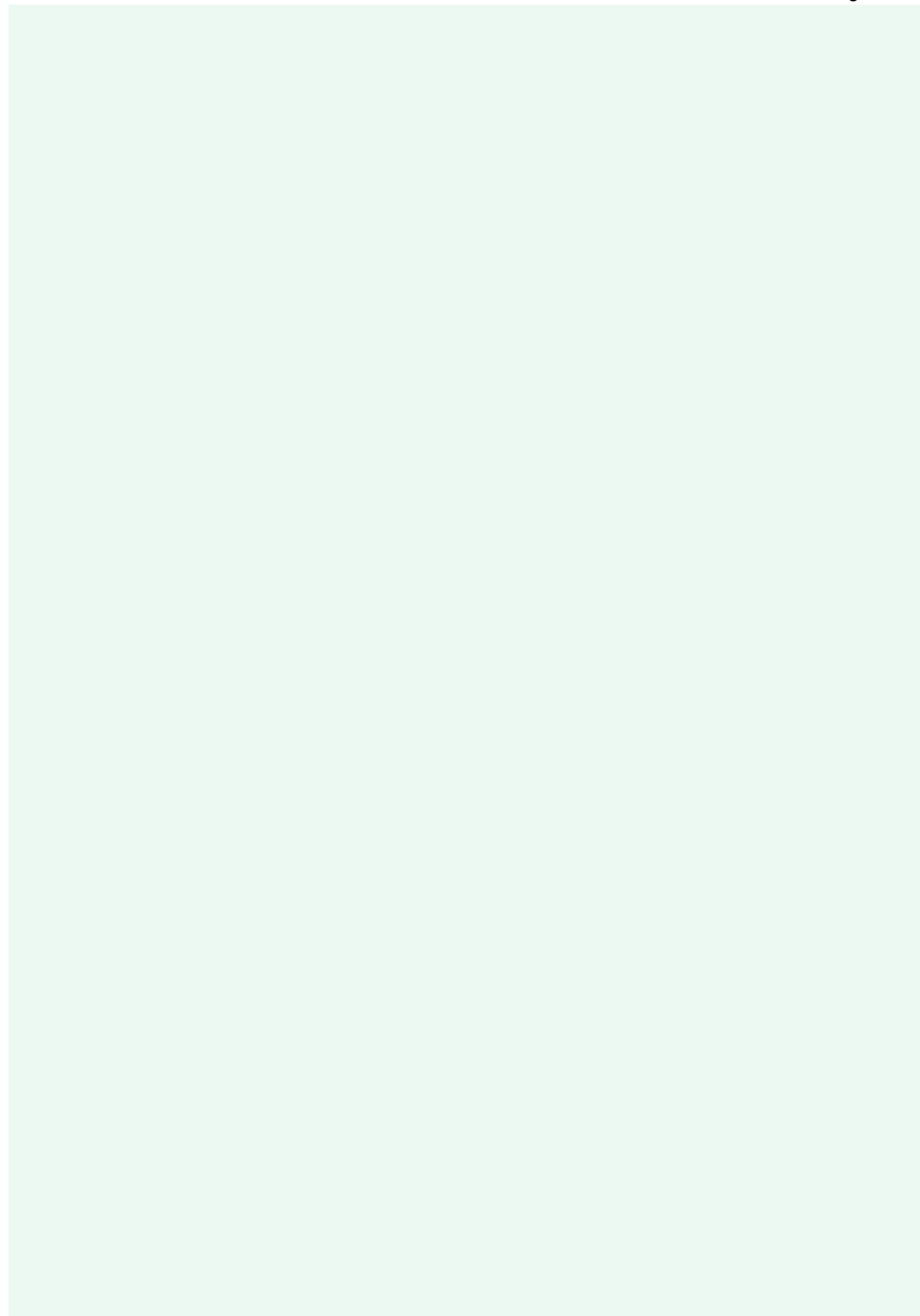
Strong MVP (8/10)

- Forecast
- Attribution
- Citizen Alerts

Winning MVP (10/10)

- Forecast
- Attribution
- Intervention Engine
- What-if Simulator
- Citizen Advisory
- Multi-city comparison

The **What-if Simulator + Intervention Agent** is where I'd spend most of the effort because that's the part judges will remember after seeing 20 other dashboards.



CPCB AQI data portal

Historical hourly AQI for all CAAQMS stations across India. Download CSV for Kolkata 2022–2024. This is your ground truth.

free **no key needed**
cpcb.nic.in/real-time-data-on-air-quality

OpenAQ API v3

Aggregates 90+ countries including India. REST API, free tier 60 req/min. Returns JSON with PM2.5, PM10, NO2, O3 per station per hour. Great for live ingestion.

free **API key (free)**
api.openaq.org/v3

NASA FIRMS (fire / thermal anomaly)

CSV downloads of thermal hotspots (biomass burning) from MODIS & VIIRS. Critical for attributing waste burning events. 375m resolution.

free **free key**
firms.modaps.eosdis.nasa.gov

Copernicus / Sentinel-5P (NO2, SO2)

Satellite NO2/SO2 data at 5.5km resolution. Download via Google Earth Engine (free academic) or Copernicus Data Space Ecosystem. Use for industrial source attribution.

free **free account**
dataspace.copernicus.eu

Open-Meteo API

Best free weather API. Historical + 7-day forecast. Wind speed, direction, temperature, humidity, boundary layer height — all the features your model needs. No key required.

free **no key needed**
api.open-meteo.com

IMD forecast API

India Meteorological Department. District-wise weather forecasts. Less clean than Open-Meteo but adds India-specific credibility to your demo.

free **registration needed**
mausam.imd.gov.in

Mapbox GL JS

50,000 free map loads/month. Best-in-class for custom heatmaps, choropleth layers, and 3D extrusions. Your GlideWay experience applies directly here.

50k free loads/mo **free key**
mapbox.com

OpenStreetMap + Overpass API

Extract industrial zones, schools, hospitals, construction sites, roads for any Indian city. Use Overpass Turbo for custom queries. Completely free.

free **no key needed**
overpass-api.de

Kolkata ward GeoJSON

KMC ward boundaries as GeoJSON. Available on data.gov.in or scrape from KMC website. Essential for your ward-level choropleth map.

free **download once**
data.gov.in (search "Kolkata ward")

AQICN / WAQI API

Real-time AQI for hundreds of Indian stations with a clean REST API. Good backup / cross-validation source. 1,000 free calls/day.

1k req/day free **free token**
aqicn.org/api

Claude API (Anthropic)

Enforcement action generation + multilingual citizen advisories. Use claude-haiku-3 for speed/cost. Free \$5 credit on signup. Your Buildora LLM work makes this easy.

\$5 free credit **API key**
console.anthropic.com

Google Colab (model training)

Free T4 GPU for training LSTM / XGBoost on CPCB historical data. Save model as .pkl and serve from FastAPI. No local GPU needed.

free GPU **Google account**
colab.research.google.com

scikit-learn + XGBoost + SHAP

XGBoost for AQI forecasting. SHAP for attribution explainability — gives you the "which source caused this spike" feature that judges specifically asked for.

Prophet (Facebook)

Excellent for seasonal AQI forecasting — handles daily/weekly/Diwali seasonality out of the box. Faster to train than LSTM for a hackathon.

free / open source

`pip install xgboost shap`

free / open source

`pip install prophet`



Frontend

Next.js 14 (App Router) + TypeScript + Tailwind CSS + Mapbox GL JS + Recharts (AQI time-series) + jsPDF (enforcement report export).

your existing stack

Backend

FastAPI (Python) for ML serving + data ingestion. Node.js/Express for LLM agent calls + WebSocket live updates. APScheduler for 15-min polling cron.

your existing stack

Database

MongoDB Atlas free tier (512 MB) — perfect for time-series AQI readings + GeoJSON ward data. Use geospatial index on station coordinates.

512 MB free

Deployment

Vercel (Next.js, free). Railway or Render (FastAPI + Node, free tier). This ensures judges can access a live URL — not localhost.

free tier

CPCB AQI data portal

Historical hourly AQI for all CAAQMS stations across India. Download CSV for Kolkata 2022–2024. This is your ground truth.

free **no key needed**

cpcb.nic.in/real-time-data-on-air-quality

OpenAQ API v3

Aggregates 90+ countries including India. REST API, free tier 60 req/min. Returns JSON with PM2.5, PM10, NO2, O3 per station per hour. Great for live ingestion.

free **API key (free)**

api.openaq.org/v3

NASA FIRMS (fire / thermal anomaly)

CSV downloads of thermal hotspots (biomass burning) from MODIS & VIIRS. Critical for attributing waste burning events. 375m resolution.

free **free key**

firms.modaps.eosdis.nasa.gov

Copernicus / Sentinel-5P (NO2, SO2)

Satellite NO2/SO2 data at 5.5km resolution. Download via Google Earth Engine (free academic) or Copernicus Data Space Ecosystem. Use for industrial source attribution.

free **free account**

dataspace.copernicus.eu

Open-Meteo API

Best free weather API. Historical + 7-day forecast. Wind speed, direction, temperature, humidity, boundary layer height — all the features your model needs. No key required.

free **no key needed**

api.open-meteo.com

IMD forecast API

India Meteorological Department. District-wise weather forecasts. Less clean than Open-Meteo but adds India-specific credibility to your demo.

free **registration needed**

mausam.imd.gov.in

Mapbox GL JS

50,000 free map loads/month. Best-in-class for custom heatmaps, choropleth layers, and 3D extrusions. Your GlideWay experience applies directly here.

50k free loads/mo **free key**

mapbox.com

OpenStreetMap + Overpass API

Extract industrial zones, schools, hospitals, construction sites, roads for any Indian city. Use Overpass Turbo for custom queries. Completely free.

free **no key needed**

overpass-api.de

Kolkata ward GeoJSON

KMC ward boundaries as GeoJSON. Available on data.gov.in or scrape from KMC website. Essential for your ward-level choropleth map.

free **download once**

data.gov.in (search "Kolkata ward")

AQICN / WAQI API

Real-time AQI for hundreds of Indian stations with a clean REST API. Good backup / cross-validation source. 1,000 free calls/day.

1k req/day free **free token**

aqicn.org/api

Claude API (Anthropic)

Enforcement action generation + multilingual citizen advisories. Use claude-haiku-3 for speed/cost. Free \$5 credit on signup. Your Buildora LLM work makes this easy.

\$5 free credit **API key**

console.anthropic.com

Google Colab (model training)

Free T4 GPU for training LSTM / XGBoost on CPCB historical data. Save model as .pkl and serve from FastAPI. No local GPU needed.

free GPU **Google account**

colab.research.google.com

scikit-learn + XGBoost + SHAP

XGBoost for AQI forecasting. SHAP for attribution explainability — gives you the "which source caused this spike" feature that judges specifically asked for.

Prophet (Facebook)

Excellent for seasonal AQI forecasting — handles daily/weekly/Diwali seasonality out of the box. Faster to train than LSTM for a hackathon.

free / open source

`pip install xgboost shap`

free / open source

`pip install prophet`



Frontend

Next.js 14 (App Router) + TypeScript + Tailwind CSS + Mapbox GL JS + Recharts (AQI time-series) + jsPDF (enforcement report export).

your existing stack

Backend

FastAPI (Python) for ML serving + data ingestion. Node.js/Express for LLM agent calls + WebSocket live updates. APScheduler for 15-min polling cron.

your existing stack

Database

MongoDB Atlas free tier (512 MB) — perfect for time-series AQI readings + GeoJSON ward data. Use geospatial index on station coordinates.

512 MB free

Deployment

Vercel (Next.js, free). Railway or Render (FastAPI + Node, free tier). This ensures judges can access a live URL — not localhost.

free tier

CPCB AQI data portal

Historical hourly AQI for all CAAQMS stations across India. Download CSV for Kolkata 2022–2024. This is your ground truth.

free **no key needed**
cpcb.nic.in/real-time-data-on-air-quality

OpenAQ API v3

Aggregates 90+ countries including India. REST API, free tier 60 req/min. Returns JSON with PM2.5, PM10, NO2, O3 per station per hour. Great for live ingestion.

free **API key (free)**
api.openaq.org/v3

NASA FIRMS (fire / thermal anomaly)

CSV downloads of thermal hotspots (biomass burning) from MODIS & VIIRS. Critical for attributing waste burning events. 375m resolution.

free **free key**
firms.modaps.eosdis.nasa.gov

Copernicus / Sentinel-5P (NO2, SO2)

Satellite NO2/SO2 data at 5.5km resolution. Download via Google Earth Engine (free academic) or Copernicus Data Space Ecosystem. Use for industrial source attribution.

free **free account**
dataspace.copernicus.eu

Open-Meteo API

Best free weather API. Historical + 7-day forecast. Wind speed, direction, temperature, humidity, boundary layer height — all the features your model needs. No key required.

free **no key needed**
api.open-meteo.com

IMD forecast API

India Meteorological Department. District-wise weather forecasts. Less clean than Open-Meteo but adds India-specific credibility to your demo.

free **registration needed**
mausam.imd.gov.in

Mapbox GL JS

50,000 free map loads/month. Best-in-class for custom heatmaps, choropleth layers, and 3D extrusions. Your GlideWay experience applies directly here.

50k free loads/mo **free key**
mapbox.com

OpenStreetMap + Overpass API

Extract industrial zones, schools, hospitals, construction sites, roads for any Indian city. Use Overpass Turbo for custom queries. Completely free.

free **no key needed**
overpass-api.de

Kolkata ward GeoJSON

KMC ward boundaries as GeoJSON. Available on data.gov.in or scrape from KMC website. Essential for your ward-level choropleth map.

free **download once**
data.gov.in (search "Kolkata ward")

AQICN / WAQI API

Real-time AQI for hundreds of Indian stations with a clean REST API. Good backup / cross-validation source. 1,000 free calls/day.

1k req/day free **free token**
aqicn.org/api

Claude API (Anthropic)

Enforcement action generation + multilingual citizen advisories. Use claude-haiku-3 for speed/cost. Free \$5 credit on signup. Your Buildora LLM work makes this easy.

\$5 free credit **API key**
console.anthropic.com

Google Colab (model training)

Free T4 GPU for training LSTM / XGBoost on CPCB historical data. Save model as .pkl and serve from FastAPI. No local GPU needed.

free GPU **Google account**
colab.research.google.com

scikit-learn + XGBoost + SHAP

XGBoost for AQI forecasting. SHAP for attribution explainability — gives you the "which source caused this spike" feature that judges specifically asked for.

Prophet (Facebook)

Excellent for seasonal AQI forecasting — handles daily/weekly/Diwali seasonality out of the box. Faster to train than LSTM for a hackathon.

free / open source

```
pip install xgboost shap
```

free / open source

```
pip install prophet
```

Frontend

Next.js 14 (App Router) + TypeScript + Tailwind CSS + Mapbox GL JS + Recharts (AQI time-series) + jsPDF (enforcement report export).

your existing stack

Backend

FastAPI (Python) for ML serving + data ingestion. Node.js/Express for LLM agent calls + WebSocket live updates. APScheduler for 15-min polling cron.

your existing stack

Database

MongoDB Atlas free tier (512 MB) — perfect for time-series AQI readings + GeoJSON ward data. Use geospatial index on station coordinates.

512 MB free

Deployment

Vercel (Next.js, free). Railway or Render (FastAPI + Node, free tier). This ensures judges can access a live URL — not localhost.

free tier

I prepared this roadmap, explain this to me

This roadmap is actually very well thought out because it optimizes for **hackathon constraints**:

- Free datasets
- Free APIs
- Your existing stack
- Fast development
- Strong demo

Let me explain every section and why it exists.

1. Air Quality Data Layer

This is your system's "ground truth".

CPCB AQI Data Portal

Purpose:

- Historical AQI records
- Training data for ML

Example:

Date	AQI
01-Jan	178
02-Jan	165
03-Jan	192

Your friend will use this data to train:

- XGBoost
- Prophet
- LSTM

models.

Without this data:

No forecasting.

OpenAQ API

Purpose:

Real-time AQI ingestion.

Instead of downloading CSVs daily:

```
JavaScript  
GET /stations
```

returns:

```
JSON  
{  
  "pm25": 102,  
  "pm10": 210,  
  "no2": 41  
}
```

You will show this live on the dashboard.

Think:

CPCB = Historical

OpenAQ = Real-time

NASA FIRMS

This is where your project becomes interesting.

Purpose:

Detect:

- Waste burning
- Biomass burning

- Fire hotspots

Example:

Suppose AQI spikes.

Normally:

AQI = 300

Nobody knows why.

But FIRMS shows:

**12 fire hotspots
within 3km**

Now AI says:

62% probability AQI spike caused by biomass burning.

This is exactly the "Source Attribution" judges want.

Sentinel-5P

Satellite pollution data.

Measures:

- NO₂
- SO₂

Example:

Heavy industrial area.

Satellite detects:

High NO₂

Your AI correlates:

**NO₂ ↑
AQI ↑
Industry nearby**

Result:

Industrial emissions likely contributing 38%.

This is a major differentiator.

Most teams won't touch satellite data.

2. Weather Layer

AQI is heavily influenced by weather.

Without weather:

Forecasting is weak.

Open-Meteo

Provides:

```
Temperature
Humidity
Wind Speed
Rainfall
Wind Direction
```

Example:

High wind:

```
AQI drops
```

Rain:

```
AQI drops sharply
```

Low wind:

```
Pollution gets trapped
```

These become ML features.

IMD API

Why included?

Judges trust IMD.

You can say:

Forecasts validated against IMD weather predictions.

Adds credibility.

3. Geospatial Layer

This is the coolest part.

Most teams build charts.

You build maps.

Mapbox GL

This powers:

Heatmaps

AQI zones

Ward Boundaries

Pollution areas

Hotspots

Industrial clusters

Construction clusters

Everything visual.

OpenStreetMap + Overpass

You can query:

```
Industries
Schools
Hospitals
Roads
Construction sites
```

Example:

Ward AQI = 280

Nearby:

```
2 hospitals
7 schools
```

AI determines:

High-risk population.

Kolkata Ward GeoJSON

This creates your city segmentation.

Without it:

Map is just a map.

With it:

```
Ward 1  
Ward 2  
Ward 3
```

Now you can say:

Ward 17 forecast AQI = 301

which matches the challenge requirement.

WAQI API

Backup source.

If OpenAQ fails:

Use WAQI.

Also useful for validation.

4. ML & AI Layer

This is where your friend works.

XGBoost + SHAP

This is probably your best choice.

Why?

Judges don't just want prediction.

They want explanation.

Example:

AQI forecast:

```
AQI tomorrow = 318
```

SHAP says:

```
Construction = 42%  
Traffic = 31%  
Humidity = 17%  
Industry = 10%
```

Now your system explains itself.

Huge advantage.

Prophet

Quick baseline model.

Benefits:

- Fast training
- Seasonality support

Good for:

```
AQI next week
```

You can compare:

```
Prophet RMSE  
XGBoost RMSE
```

during judging.

Claude API

Used for:

Enforcement Agent

Input:

```
Ward AQI = 312  
Cause = Construction
```

Output:

```
Suspend construction  
Increase dust suppression  
Deploy inspectors
```

Citizen Advisory Agent

Input:

```
AQI = 325
```

Output:

English
Hindi
Bengali
Tamil

advisories.

This directly addresses the multilingual requirement.

5. Your Tech Stack

This roadmap intentionally matches your experience.

Frontend

Next.js

Dashboard

Tailwind

UI

Recharts

AQI graphs

Mapbox

Maps

You already know all of these.

Backend

FastAPI

ML serving

Example:

```
Python
POST /predict
```

returns

```
JSON
{
  "aqi": 312
```

```
}
```

Node.js

Everything else:

- auth
- APIs
- polling
- WebSockets

This separation is smart.

MongoDB

Stores:

```
JSON
{
  stationId: "kol_01",
  aqi: 198,
  timestamp: "..."
```

Also:

- ward data
 - forecasts
 - advisories
-

The Most Important Thing Missing

Your roadmap has data sources and tech.

What wins the hackathon is:

Intervention Intelligence Engine

Add one box:

```
AQI Forecast
  ↓
Source Attribution
  ↓
Intervention Simulator
  ↓
Citizen Advisory
```

Example:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Ward 17 AQI tomorrow = 310

AI says:

Reduce truck movement by 25%

Simulation:

AQI becomes 276

Now you're not predicting pollution.

You're helping cities reduce pollution.

That is exactly what the judges are asking for.

If I Were Building This

Day 1

- Next.js Dashboard
- Mapbox
- OpenAQ Integration
- MongoDB

Day 2

- XGBoost Forecasting
- SHAP Explanations

Day 3

- Intervention Engine
- Citizen Advisory Agent
- Demo Story

The feature I'd spend the most time polishing is the **What-If Simulator + Intervention Recommendation Engine**, because that is the difference between a good project and a winning project.

setup the project folder structure and explain it to me

Since you're targeting a hackathon, don't over-engineer with microservices initially.

I would structure it as:

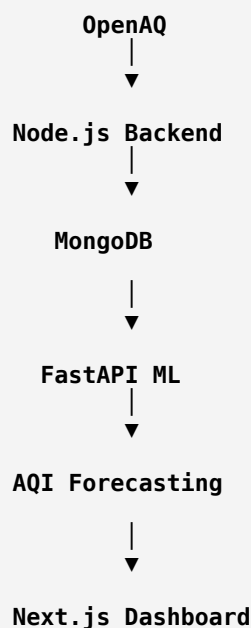
```
aeris-ai/
├── frontend/      # Next.js Dashboard
└── backend/       # Node.js APIs
```

```

├── ml-service/           # FastAPI + ML models
├── datasets/            # Raw datasets
├── docs/                # Architecture + PPT assets
└── docker-compose.yml

```

High Level Architecture



Frontend Structure

```

frontend/
├── src/
├── app/
│   ├── dashboard/
│   ├── map/
│   ├── forecast/
│   ├── interventions/
│   ├── advisories/
│   └── compare-cities/
├── components/
│   ├── layout/
│   │   ├── Sidebar.tsx
│   │   ├── Navbar.tsx
│   │   └── PageHeader.tsx
│   ├── maps/
│   │   ├── AQIMap.tsx
│   │   ├── WardHeatmap.tsx
│   │   ├── SourceLayer.tsx
│   │   └── RiskLayer.tsx
│   ├── charts/
│   │   ├── AQITrendChart.tsx
│   │   ├── SourcePieChart.tsx
│   │   ├── ForecastChart.tsx
│   │   └── SHAPChart.tsx
│   ├── forecast/
│   │   ├── ForecastCard.tsx
│   │   └── ForecastTable.tsx
│   └── interventions/

```

```

├── InterventionCard.tsx
├── ImpactSimulator.tsx
├── ActionTimeline.tsx
├── advisories/
│   ├── AdvisoryCard.tsx
│   └── LanguageSelector.tsx
├── lib/
│   ├── api.ts
│   ├── mapbox.ts
│   └── constants.ts
├── hooks/
├── store/
│   └── useAQIStore.ts
├── types/
└── public/

```

Why this structure?

You have 5 major modules:

Dashboard

/dashboard

Shows:

- Current AQI
- City overview
- Active alerts

Map

/map

Shows:

- AQI heatmap
- Pollution hotspots
- Ward boundaries

This will be the hero page.

Forecast

/forecast

Shows:

- 24h AQI
- 48h AQI
- 72h AQI

Prediction charts.

Interventions

/interventions

Most important module.

Shows:

Reduce truck traffic 25%
Expected AQI improvement:
-18 points

Advisories

/advisories

Citizen-facing recommendations.

Backend Structure

Node.js handles:

- ingestion
- auth
- maps
- MongoDB

```
backend/
├── src/
├── config/
│   ├── db.ts
│   └── env.ts
├── modules/
│   └── aqi/
```

```

├── aqi.controller.ts
├── aqi.service.ts
├── aqi.routes.ts
├── stations/
├── forecasts/
├── interventions/
├── advisories/
├── maps/
├── jobs/
│   ├── openaq.job.ts
│   ├── weather.job.ts
│   └── satellite.job.ts
├── services/
│   ├── openaq.service.ts
│   ├── weather.service.ts
│   ├── firms.service.ts
│   └── sentinel.service.ts
├── websocket/
└── server.ts

```

Why Jobs Folder?

Every 15 minutes:

OpenAQ
Weather
FIRMS

are fetched automatically.

jobs/

handles that.

ML Service Structure

Your friend owns this.

```

ml-service/
├── app/
├── models/
│   ├── xgboost.pkl
│   ├── prophet.pkl
│   └── scaler.pkl
├── training/
│   ├── train_xgboost.py
│   ├── train_prophet.py
│   └── feature_engineering.py
├── explainability/
│   └── shap_analysis.py

```

```

├── routes/
│   ├── forecast.py
│   ├── attribution.py
│   └── simulation.py
├── services/
│   ├── forecast_service.py
│   ├── attribution_service.py
│   └── simulation_service.py
└── main.py

```

Most Important ML Endpoints

Forecast

POST /forecast

returns:

```

JSON
{
  "aqi": 312,
  "confidence": 0.92
}

```

Attribution

POST /attribution

returns:

```

JSON
{
  "traffic": 31,
  "construction": 42,
  "industry": 19,
  "other": 8
}

```

Simulation

POST /simulate

Input:

```

JSON
{
  "traffic_reduction": 20,
  "construction_reduction": 30
}

```

Output:

```
JSON
{
  "aqi_before": 312,
  "aqi_after": 274
}
```

This endpoint will impress judges.

Database Structure

MongoDB Collections

stations

```
JSON
{
  stationId,
  city,
  lat,
  lng
}
```

aqi_readings

```
JSON
{
  stationId,
  pm25,
  pm10,
  no2,
  aqi,
  timestamp
}
```

forecasts

```
JSON
{
  wardId,
  forecastAQI,
  generatedAt
}
```

interventions

```
JSON
{
  wardId,
  recommendation,
  impact
}
```

advisories

```
JSON
```

```
{
  wardId,
  language,
  content
}
```

Hackathon MVP Folder Scope

For 48 hours, I would focus on only:

```
frontend
├── dashboard
├── map
├── forecast
└── interventions

backend
├── ingestion
└── APIs

ml-service
├── forecast
├── attribution
└── simulation
```

Skip authentication, RBAC, notifications, and complex DevOps.

The winning flow should be:

```
AQI Data
  ↓
Forecast
  ↓
Attribution
  ↓
What-If Simulation
  ↓
AI Recommendation
  ↓
Map Visualization
```

That directly maps to the judging criteria and demonstrates a complete "signal → decision → intervention" pipeline rather than just another analytics dashboard.

can i use nextjs for backend as well or nodejs will be better

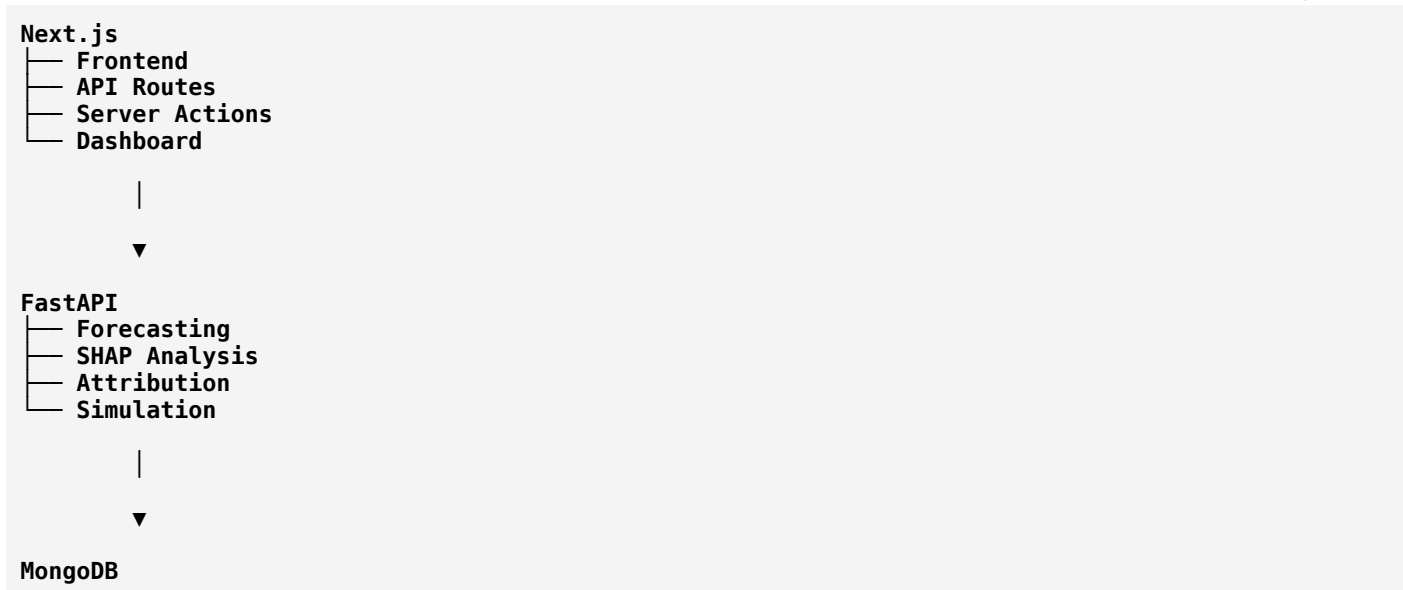
For a **hackathon**, I would strongly recommend:

 **Next.js + FastAPI**

Skip the separate Node.js backend.

Why?

Your architecture becomes:



Only **2 services** instead of **3 services**.

Option 1 (Recommended)

Next.js handles:

- UI
- API routes
- Data ingestion
- Cron jobs
- MongoDB operations
- Authentication (if needed)

Example:

```

/app/api/aqi/route.ts
/app/api/stations/route.ts
/app/api/advisories/route.ts
/app/api/interventions/route.ts
  
```

You already know Next.js very well from:

- Buildora
- Merchanta AI
- Registration Editor
- Dashboard projects

So development will be much faster.

Option 2

Next.js + Express + FastAPI

```
Next.js
  ↓
Express
  ↓
FastAPI
```

Problem:

More complexity.

You'll spend time debugging:

```
CORS
Ports
Proxy
Deployment
Environment Variables
```

instead of building features.

For Hackathon Judges

They don't care whether:

```
Node.js
Express
NestJS
Next.js API Routes
```

is used.

They care about:

- Forecast accuracy
- Geospatial intelligence
- Source attribution
- Intervention recommendations
- UX

What I Would Build

```
aeris-ai/
frontend/
├── app/
└── api/
```

```

|   |   | aqi/
|   |   | forecast/
|   |   | interventions/
|   |   | advisories/
|   |   | stations/
|   | components/
|   | lib/
|   | hooks/
|   | types/
| ml-service/
|   | models/
|   | training/
|   | routes/
|   | services/
|   | main.py

```

Communication Flow

User opens dashboard

```
Next.js
```

↓

Next.js requests forecast

```

TypeScript
fetch("http://fastapi:8000/forecast")

```

↓

FastAPI returns

```

JSON
{
  "aqi": 312,
  "confidence": 0.91
}

```

↓

Next.js renders chart

MongoDB Access

I would let **Next.js** own MongoDB.

```

Next.js
  ↓
MongoDB

```

FastAPI only handles ML.

This keeps responsibilities clean.

Deployment

Super simple:

Vercel

Next.js

Render/Railway

FastAPI

MongoDB Atlas

Database

Only 3 deployed pieces.

Final Recommendation

For your team and timeline:

- ✓ Next.js (Frontend + Backend APIs)
- ✓ FastAPI (ML Service)
- ✓ MongoDB Atlas

Do **not** add Express unless you discover a specific requirement that Next.js API routes cannot handle.

For this project, Next.js API routes are more than enough, and you'll probably save 6–10 hours of development time that can instead go into the **What-If Simulator** and **Intervention Intelligence Engine**, which are much more likely to influence judging.

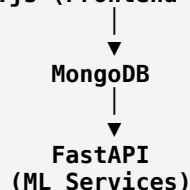
provide me with the final project structure now

For **AERIS AI**, I would finalize the structure like this.

This is production-grade enough to impress judges, but still realistic for a hackathon.

Final Architecture

Next.js (Frontend + Backend APIs)



Project Structure

```

aeris-ai/
├── frontend/                # Next.js App
├── ml-service/              # FastAPI ML Service
├── datasets/                # Training + GeoJSON Data
├── docs/                    # Architecture + PPT Assets
├── .env.example
├── docker-compose.yml
├── README.md
└── architecture.png

```

Frontend (Next.js)

```

frontend/
├── src/
├── app/
│   ├── page.tsx             # Landing Page
│   ├── dashboard/
│   │   └── page.tsx
│   ├── map/
│   │   └── page.tsx
│   ├── forecast/
│   │   └── page.tsx
│   ├── interventions/
│   │   └── page.tsx
│   ├── advisories/
│   │   └── page.tsx
│   ├── compare-cities/
│   │   └── page.tsx
│   └── api/
│       ├── aqi/
│       │   └── route.ts
│       ├── stations/
│       │   └── route.ts
│       ├── forecast/
│       │   └── route.ts
│       ├── interventions/
│       │   └── route.ts
│       ├── advisories/
│       │   └── route.ts
│       └── sync/
│           └── route.ts
└── components/
    └── layout/

```

- Sidebar.tsx
- Navbar.tsx
- PageHeader.tsx

- maps/
 - AQIMap.tsx
 - WardHeatmap.tsx
 - SourceAttributionLayer.tsx
 - PollutionHotspots.tsx
 - VulnerabilityLayer.tsx

- charts/
 - AQITrendChart.tsx
 - ForecastChart.tsx
 - SHAPChart.tsx
 - SourceBreakdownChart.tsx
 - ComparisonChart.tsx

- forecast/
 - ForecastCard.tsx
 - ForecastSummary.tsx
 - ForecastTable.tsx

- interventions/
 - InterventionCard.tsx
 - InterventionTimeline.tsx
 - WhatIfSimulator.tsx
 - ImpactEstimator.tsx

- advisories/
 - AdvisoryCard.tsx
 - LanguageSelector.tsx
 - HealthRiskCard.tsx

- common/
 - Loading.tsx
 - EmptyState.tsx
 - MetricCard.tsx
 - StatusBadge.tsx

lib/

- mongodb.ts
- mapbox.ts
- constants.ts
- api.ts
- utils.ts

hooks/

- useAQI.ts
- useForecast.ts
- useInterventions.ts
- useMap.ts

services/

- openaq.service.ts
- weather.service.ts

```

├── firms.service.ts
├── sentinel.service.ts
├── ml.service.ts
├── store/
│   └── useAQIStore.ts
├── types/
│   ├── aqi.ts
│   ├── forecast.ts
│   ├── map.ts
│   └── advisory.ts
├── public/
│   ├── geojson/
│   │   └── kolkata-wards.geojson
│   └── icons/
└── package.json

```

ML Service (FastAPI)

This is your friend's zone.

```

ml-service/
├── app/
├── routes/
│   ├── forecast.py
│   ├── attribution.py
│   ├── simulation.py
│   └── advisory.py
├── services/
│   ├── forecast_service.py
│   ├── attribution_service.py
│   ├── simulation_service.py
│   └── advisory_service.py
├── training/
│   ├── train_xgboost.py
│   ├── train_prophet.py
│   ├── feature_engineering.py
│   └── evaluate.py
├── models/
│   ├── xgboost.pkl
│   ├── prophet.pkl
│   ├── scaler.pkl
│   └── label_encoder.pkl
├── explainability/
│   └── shap_analysis.py

```

```

├── feature_importance.py
├── schemas/
│   ├── forecast.py
│   ├── attribution.py
│   └── simulation.py
├── main.py
├── requirements.txt
└── Dockerfile

```

Dataset Folder

```

datasets/
├── aqi/
│   ├── cpcb_historical.csv
│   └── openaq_backup.csv
├── weather/
│   └── openmeteo_data.csv
├── satellite/
│   ├── firms_hotspots.csv
│   └── sentinel_pollution.csv
├── geojson/
│   ├── kolkata_wards.geojson
│   ├── hospitals.geojson
│   ├── schools.geojson
│   └── industries.geojson
└── processed/

```

Docs Folder

```

docs/
├── architecture/
│   └── system-design.png
├── presentation/
│   └── final-ppt.pptx
├── demo/
│   └── script.md
└── screenshots/

```

The 4 Core Features You Must Finish

If time becomes short, prioritize only these:

1. AQI Forecasting

```
AQI today
↓
AQI after 24h
↓
AQI after 48h
↓
AQI after 72h
```

2. Source Attribution

```
Traffic      34%
Construction 29%
Industry     25%
Burning      12%
```

3. What-If Simulator

```
Reduce Traffic 20%

AQI:
312 → 284
```

This is the wow feature.

4. Intervention Recommendation Engine

```
Priority 1:
Suspend construction

Impact:
-18 AQI

Priority 2:
Restrict diesel trucks

Impact:
-12 AQI
```

If you execute this structure properly, you'll cover **all five major areas mentioned in the problem statement**:

- ✓ AQI Forecasting
- ✓ Source Attribution
- ✓ Enforcement Intelligence
- ✓ Citizen Advisory
- ✓ Multi-City Intelligence

while keeping the codebase manageable for a hackathon team of two.